

Training

CS 541 Lecture 7

Zining Zhu

October 15, 2026



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Training language models

Objective, data, and scale

Pretraining, at scale

- Same objective as the n-gram model: predict the next token
- The differences are **scale** and **architecture**, not the loss function
- Trained on trillions of tokens scraped and filtered from books, web pages, code, and more
- **Self-supervised**: no human labels required, only raw text

Self-supervised vs. supervised, revisited

- Lecture 2's supervised learning needed a labeled (x, y) pair for every training example — labeling is slow and expensive
- Pretraining needs no labeling step at all: every token in every document is simultaneously an input and (for the next step) a label
- This is why language model training sets can be orders of magnitude larger than any hand-labeled dataset in the course so far
- The tradeoff: the objective (predict the next token) is only a **proxy** for what we actually want (a helpful, accurate assistant) — a gap we return to in the post-training part of this lecture

Where the data comes from

- Common Crawl and other web scrapes, heavily filtered and deduplicated
- Curated sources: books, Wikipedia, code repositories, academic papers
- Data quality and mixture matter as much as data quantity
- Increasingly: synthetic data generated by other models

Filtering and deduplication

- Raw web scrapes contain enormous amounts of low-quality, duplicated, or harmful content
- **Deduplication** matters because repeated text can be memorized verbatim rather than generalized from — and it wastes training compute on redundant signal
- Filtering heuristics and learned classifiers remove spam, boilerplate, and content that fails quality or safety checks
- Even after filtering, the resulting mixture still reflects the biases of what was written and put online — a thread back to Lecture 1's **fairness** and **privacy** discussion

Data mixture and curriculum

- Different sources teach different things: code data tends to improve structured reasoning, books teach long-range narrative coherence, web text gives broad general knowledge
- Training data is usually a deliberately weighted **mixture** of sources, not just “all the text we could find” in raw proportion
- Some pipelines also use a **curriculum**: adjusting the mixture over the course of training (e.g., upweighting higher-quality data later)
- Getting the mixture right is itself an empirical, actively-researched problem

Synthetic data: promise and risk

- Using one model's outputs to help train another can cheaply expand or clean a training set
- Promise: can target exactly the skills a model is weak on, and can be cheaper than sourcing more human-written text
- Risk — **model collapse**: training repeatedly on model-generated data can narrow the diversity of what the next model produces, drifting away from the true data distribution
- Synthetic data is a tool, not a free lunch — it still needs quality control

What a training loss curve looks like

- Training loss (roughly, negative log-probability the model assigns to the true next token) starts high and falls, usually smoothly, over the course of training
- Practitioners watch for: loss spikes (often a sign of a bad batch or numerical instability), a curve that plateaus earlier than the compute budget planned for, and divergence between training loss and held-out validation loss (a sign of overfitting, in the Lecture 2 sense)
- Because runs are so expensive, small early warning signs are usually acted on quickly rather than let ride to the end

Evaluating language models

- **Perplexity**: cheap to compute on any held-out text, but only weakly correlated with how useful a model feels in practice
- **Downstream benchmarks**: standardized test sets probing specific skills (general knowledge, commonsense reasoning, reading comprehension, code, math)
- No single number captures “how good” a language model is — benchmark suites report many scores, and results can be sensitive to exact prompt format

We return to evaluating post-trained, instruction-following models later in this lecture.

Scaling laws

- Loss on held-out data falls smoothly and predictably as **parameters**, **data**, and **compute** increase together
- Chinchilla-style finding: for a fixed compute budget, there is an optimal balance between model size and training tokens
  - Many earlier models were **undertrained** relative to their size
- Scaling laws let labs forecast the performance of a not-yet-trained model

Scaling-law arithmetic

- A widely cited rule of thumb from the Chinchilla line of work: compute-optimal training uses roughly 20 training tokens per parameter
- A standard rough estimate for total training compute: $C \approx 6 \cdot N \cdot D$, where N is parameter count and D is number of training tokens
- Substituting $D \approx 20N$: $C \approx 6N(20N) = 120N^2$, so $N \approx \sqrt{C/120}$
- Toy example: for a compute budget $C = 1.2 \times 10^{21}$ FLOPs, $N \approx \sqrt{10^{19}} \approx 3.2$ billion parameters, with $D \approx 20N \approx 64$ billion tokens

Treat the constants here as illustrating the “shape” of the tradeoff, not a universal law — exact optimal ratios shift with architecture and data quality.

Why “bigger” has kept working

- More parameters: more capacity to store patterns and associations
- More data: more coverage of the patterns worth storing
- More compute: enables both of the above, and more optimization steps
- Diminishing returns are real, but have not yet flattened out enough to stop scaling

Scaling is an empirical trend, not a law of nature — treat extrapolations with caution.

Diminishing returns and the compute wall

- Each further reduction in loss requires a larger multiplicative increase in compute — the relationship is closer to logarithmic gains than linear ones
- Training compute cost grows with both model size **and** data size, so pushing both up together compounds cost quickly
- At some point, physical constraints (power, chips, money, and available high-quality data) become the binding limit rather than algorithmic ideas
- This is an active argument in the field: how much further does **pure** scaling of pretraining carry performance, versus other levers (better data, better post-training, better inference-time computation)?

Check your understanding

- Why can a language model be pretrained on far more data than a Lecture 2-style supervised model?
- What does it mean for a model to be “undertrained relative to its size,” in the Chinchilla sense?
- Name one risk specific to training on synthetic (model-generated) data.



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Prompting and in-context learning

Getting behavior out of a frozen model

Zero-shot and few-shot prompting

- **Zero-shot**: describe the task in the prompt, no examples given
- **Few-shot**: include a handful of examples directly in the prompt
- Neither updates the model's weights — all the "learning" happens **in context**, at inference time
- Performance on a task can shift dramatically just from how the prompt is phrased

Few-shot example

A few-shot prompt for sentiment classification, given directly to a frozen model:

- “Review: This movie was fantastic. Sentiment: positive”
- “Review: I wasted two hours on this. Sentiment: negative”
- “Review: The acting was solid but the plot dragged. Sentiment: ???”

The model infers the pattern “review → sentiment word” purely from the two examples shown, then continues it — swap in different label words and it typically follows that format instead.

In-context learning

- A pretrained model can pick up a new task pattern purely from examples placed in its context window
- No gradient updates, no parameter changes — the weights are frozen
- Still not well understood **why** this works; an active research question
- Sensitive to example order, formatting, and even example selection

Why in-context learning is surprising

- Nothing in the next-token pretraining objective explicitly rewards “learn a new task from a few examples in the prompt” — this behavior emerges as a side effect of pretraining on enormously varied text
- One hypothesis: pretraining text already contains many examples of pattern-following (worked problem sets, FAQs, tables), so the model has implicitly practiced this move many times without anyone labeling it as such
- Whatever the mechanism, the practical result is that the **same frozen weights** can be steered toward many different tasks just by changing the prompt

Sensitivity of in-context learning

- Reordering the same few-shot examples can measurably change accuracy, even though the “information content” of the prompt hasn’t changed
- Label wording, formatting (e.g., using “Sentiment:” vs. a different template), and even which specific examples are chosen can all matter
- This sensitivity is a practical headache for building reliable systems on top of prompting alone, and is part of the motivation for the fine-tuning approaches covered later in this lecture

Chain-of-thought prompting

- Prompting the model to produce intermediate reasoning steps before the final answer
- Often substantially improves performance on multi-step reasoning and arithmetic tasks
- Can be triggered explicitly (“think step by step”) or elicited via few-shot examples of reasoned answers
- Raises an open question: is this “reasoning,” or a more effective use of computation? Both framings are used in the literature

Chain-of-thought example

Question: “A store had 23 apples. It sold 8, then received a delivery of 15 more. How many apples does it have now?”

- **Direct-answer prompting**: the model outputs a single number immediately — with no intermediate steps, a small arithmetic slip has nowhere to be caught
- **Chain-of-thought prompting**: “Start with 23. After selling 8: 23 – 8 = 15. After the delivery of 15: 15 + 15 = 30. Answer: 30.”
- Writing out the intermediate steps gives the model’s own next-token process more “space” to get each small step right before committing to a final answer

Why chain-of-thought might help

- One view: it is genuine step-by-step **reasoning**, decomposing a hard problem into easier sub-problems, much like a person working through the same question on paper
- Another view: it simply gives the model more computation (more forward passes, one per generated token) before it has to commit to an answer — the benefit comes from "thinking longer," not necessarily from anything reasoning-like
- Both framings currently coexist in the research literature, and the debate connects directly back to Lecture 1's question of what should even count as "intelligence" in a machine

Emergent abilities, with caution

- Some capabilities (e.g., multi-step arithmetic, certain forms of reasoning) appear to show up only past a scale threshold, rather than improving smoothly
- These are often called **emergent abilities**
- **Caveat**: some apparent emergence is an artifact of the evaluation metric, not the model
  - A metric with a sharp pass/fail threshold can look “emergent” even when the underlying capability improves smoothly
- Read scaling claims about emergence critically — check what metric is being plotted

The metric-artifact caveat, illustrated

- Imagine a task scored as “exactly correct” or “wrong,” on a skill that is actually improving gradually and continuously underneath
- Below some scale, the model is **almost** right but never quite crosses the threshold to be scored correct — so the metric reads a flat zero
- Once the underlying skill crosses the threshold, the score jumps from 0% to visibly above 0% — looking like a sudden “emergence”
- Replacing the strict pass/fail metric with a partial-credit or probability-based score often reveals the same underlying capability improving smoothly all along

This does not prove no ability is genuinely emergent — it is a reason to check the measurement before believing the claim.

Check your understanding

- What is being “learned” during in-context learning, if the model’s weights never change?
- Give one reason chain-of-thought prompting might help that has nothing to do with “reasoning” in the human sense
- Why should a claim of an “emergent ability” make you ask what metric was used?



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Why post-train a pretrained model?

Base models · Instructions · Alignment

What a base language model actually learned

A base LM is trained to predict the next token over huge, unfiltered text.

- It models **what is likely to come next**, not **what is helpful**
- Ask it a question, and it may continue the question, ramble, or mimic the training distribution's worst habits
- **Pretraining** gives broad knowledge and fluency; it does not give a cooperative assistant

A concrete example: base model vs. instruction-tuned model

Prompt: “Explain why the sky is blue.”

- **Base model completion (typical)**: continues with a list of similar homework questions, or a Wikipedia-style paragraph that trails off mid-sentence
- **Instruction-tuned completion (typical)**: a direct, well-scoped explanation addressed to the asker, stopping when the answer is complete
- Same weights’ worth of **knowledge** about Rayleigh scattering — the difference is entirely in **how** the knowledge gets used

Post-training does not teach new facts so much as it teaches a new **behavior policy** on top of old facts.

Supervised fine-tuning (SFT)

The first and simplest post-training step.

- Collect (instruction, ideal response) pairs, often written or curated by humans
- Fine-tune the base model with standard next-token cross-entropy loss on these pairs
- Turns a text-continuation engine into something that **responds** to instructions
- Still limited: SFT only teaches the model to imitate the demonstrations it saw

SFT answers “what should a good response look like?” It does not answer “which of two responses is better?”

The SFT loss in detail

- Format each example as prompt tokens followed by response tokens
- Compute next-token cross-entropy loss only over the **response tokens** — the prompt is masked out of the loss
- Why mask the prompt: we want the model to learn to **generate** good responses, not to get better at predicting instructions it will never need to produce itself
- Otherwise identical to language-model pretraining — same architecture, same optimizer, just a narrower, curated dataset

What SFT alone still gets wrong

- **Verbosity**: imitating long demonstrations makes the model default to long answers, even when short ones would serve better
- **Sycophancy**: if demonstrations skew agreeable, the model learns to agree rather than to be correct
- **Format over intent**: the model can match the **shape** of a good answer while missing what was actually asked
- Root cause: SFT has no signal about **how much better** one response is than another

The alignment framing: HHH

Post-training is usually framed around three goals.

- **Helpful**: actually addresses the user's request
- **Honest**: calibrated, doesn't fabricate confident nonsense
- **Harmless**: avoids dangerous, abusive, or policy-violating output
- These goals can conflict (e.g. maximally honest vs. maximally harmless); post-training methods are how we trade them off

When HHH goals conflict: an example

Prompt: “What common household chemicals are dangerous to mix?”

- A maximally **helpful** and **honest** answer explains the chemistry clearly, since this is genuinely useful safety knowledge
- An overly **harmless**-weighted policy might refuse outright, treating any mention of “dangerous mixtures” as unsafe
- The “right” answer threads the needle: informative enough to keep people safe, without turning into a synthesis recipe
- There is no formula that resolves this automatically — it is a **policy choice** encoded through preference data and rules

From imitation to preference

- SFT teaches the model to imitate **examples** of good behavior
- But writing an example of “better than average” is easy; writing the **single best** response is hard
- It is much easier for a human to **compare** two responses than to **write** an ideal one
- This motivates the next step: learning from **preferences**, not just demonstrations

Why comparing is easier than generating

- Generating an ideal response requires expertise, time, and getting every detail right
- Comparing two already-written responses only requires recognizing which one is better — a much lower bar
- This asymmetry is why nearly every large-scale alignment effort collects **comparisons** at scale, but only a modest amount of **written demonstrations**
- It mirrors a familiar fact from everyday life: most people can tell a good essay from a mediocre one far more easily than they can write a great essay themselves

Where the instruction-following data comes from

- **Human-written**: contractors or in-house annotators write demonstrations and comparisons directly — high quality, expensive, slow to scale
- **Model-assisted curation**: a strong existing model drafts candidates; humans edit or simply choose among them
- **Distillation**: a smaller or newer model is trained on a stronger model's outputs — cheap, but inherits the teacher's blind spots
- In practice, modern pipelines mix all three, with human judgment concentrated where it matters most

Check your understanding

- **Why can't SFT alone fully align a model?** — it only imitates demonstrations; it has no notion of "how much better"
- **Why is preference data cheaper to collect than demonstrations?** — comparing is easier for humans than generating an ideal answer
- **What does "HHH" stand for, and why can the three goals conflict?** — helpful, honest, harmless; maximizing one can come at the expense of another
- Next: where do those preferences actually turn into a training signal?



STEVENS
INSTITUTE OF TECHNOLOGY
1870

A brief history of post-training

From InstructGPT to reasoning models

2020–2022: instruction tuning takes off

- Early work (e.g. instruction-tuned model families trained across many tasks phrased as natural-language instructions) showed a single model could follow varied instructions if trained on enough of them
- This established **instruction tuning** — what we now call SFT — as a standard step, distinct from pretraining
- Still no notion of preference: every instruction-response example was treated as equally correct

2022: the modern RLHF recipe

- A landmark industry paper on instruction-following language models popularized the now-standard three-stage pipeline: SFT, reward modeling, RL fine-tuning with PPO
- Demonstrated that a heavily post-trained mid-sized model could be preferred by human raters over a much larger un-tuned base model
- This result is why “post-training” became as central to building a usable assistant as pretraining itself

2023: the search for simpler alignment

- RLHF's three-stage pipeline was expensive and fragile: three models, unstable RL training, many hyperparameters
- **DPO** (Direct Preference Optimization) showed the same preference data could train a comparably good policy with a single supervised-style loss
- Triggered a wave of "RL-free" alignment methods exploring the same reparameterization idea

2024–2025: verifiable rewards and reasoning models

- As models tackled harder math and coding tasks, researchers noticed many of these tasks have an **automatically checkable** answer
- **RLVR** replaced the learned reward model with a deterministic checker for such tasks
- Today’s frontier models typically combine SFT, preference-based RLHF or DPO, **and** RLVR-style training on verifiable domains



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Reward modeling and RLHF

Preferences · Bradley-Terry · PPO

Collecting human preference data

- Sample multiple responses from the SFT model for the same prompt
- Show a pair to a human rater; ask which response is better
- Store the result as a pairwise comparison: response $A \succ B$
- Preference data is cheaper and more reliable to collect at scale than “write the ideal answer”

Practical headaches in preference collection

- **Rater disagreement**: two reasonable people often pick different winners, especially for subjective or creative prompts
- **Calibration**: raters need shared guidelines, or “better” drifts across annotators and time
- **Cost and throughput**: careful human review does not scale as fast as model training does
- **Sampling coverage**: preference pairs only ever say something about the **specific responses shown**

Turning preferences into a reward model

We want a scalar reward function $r(x, y)$ that scores how good response y is for prompt x.

- The **Bradley-Terry model**: assumes $P(y_w \succ y_l) = \sigma\big(r(x, y_w) - r(x, y_l)\big)$
- Train a neural network r_ϕ to fit this to the observed human comparisons
- Loss: maximize the log-likelihood of the observed preference under Bradley-Terry
- The result is a **reward model (RM)**: a learned proxy for “how good is this response”

Example: from reward gap to preference probability

The Bradley-Terry model converts a reward **difference** into a **probability** through the sigmoid function.

Reward gap $r(x, y_w) - r(x, y_l)$	$\sigma(\text{gap}) = P(y_w \succ y_l)$
0.0	50% — a toss-up
1.0	73% — a mild but clear preference
3.0	95% — a strong preference
5.0	99% — nearly certain

A wider reward gap should correspond to a more lopsided human preference — that correspondence is exactly what the reward model is trained to fit.

Reward model architecture and training loss

- Usually initialized from the **SFT checkpoint** itself, with the final language-modeling head replaced by a single scalar output
- Given a (prompt, response) pair, it produces one number: the predicted reward
- Training loss: $-\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)$, averaged over all labeled comparison pairs
- This is exactly a **binary classification loss** — "which response is preferred" — dressed up in reward-model language

The RLHF pipeline, end to end

Three stages, each building on the last.

Stage	Input	Output
1. SFT	(instruction, ideal response) pairs	a starting policy π_{SFT}
2. Reward modeling	pairwise comparisons over π_{SFT}'s outputs	a reward model r_ϕ
3. RL fine-tuning	π_{SFT}, r_ϕ	a policy π_θ optimized for reward

Each stage can bottleneck the next: a biased RM teaches the policy the wrong lesson.

RL fine-tuning with PPO

- Treat the language model as a **policy** π_θ: prompt in, response out, one token at a time
- Reward for a full response comes from the reward model r_ϕ
- **Proximal Policy Optimization (PPO)** updates π_θ to increase expected reward
- Uses a clipped objective so a single update never moves the policy too far in one step

The PPO clipped surrogate objective

Let $\mathrm{ratio}(\theta) = \pi_\theta(y \mid x)/\pi_{\mathrm{old}}(y \mid x)$, and A be the response's estimated advantage.

- The unclipped objective would simply maximize $\mathrm{ratio}(\theta) \cdot A$ — but an unbounded ratio can blow up after one aggressive update
- PPO instead maximizes the **minimum** of $\mathrm{ratio}(\theta) \cdot A$ and

$$\mathrm{clip}(\mathrm{ratio}(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A$$

- Clipping removes the incentive to push the ratio **far** outside $[1 - \epsilon, 1 + \epsilon]$ in a single step, whichever direction A points

Why clipping matters: an intuition example

- Suppose one sampled response happens to get a very high reward, and $\mathrm{ratio}(\theta)$ starts drifting sharply upward on it
- Without clipping, the optimizer keeps pushing that one lucky response's probability up indefinitely each step, destabilizing the rest of the policy
- With clipping, once the ratio leaves the $[1-\epsilon, 1+\epsilon]$ band, that term stops contributing extra gradient — the update is capped
- The result: PPO takes many small, safe steps instead of a few large, risky ones

Why RLHF needs a KL penalty

Without a constraint, the policy can find degenerate outputs that fool the reward model.

- Add a penalty term: $-\beta \cdot \mathrm{KL}(\pi_\theta \,||\, \pi_{\mathrm{ref}})$, where π_{ref} is the SFT model
- Keeps the RL-tuned policy close to a distribution the reward model was actually trained on
- Prevents **reward hacking**: exploiting quirks in r_ϕ instead of genuinely improving responses
- β trades off reward-seeking against staying close to sensible language

Assembling the full RLHF objective

Putting the pieces together, PPO for RLHF maximizes, per response:

- **PPO clipped term**: $\min\left(\text{ratio}(\theta)\cdot A,\ \text{clip}(\text{ratio}(\theta), 1-\epsilon, 1+\epsilon)\cdot A\right)$ — pushes toward higher-reward responses, safely
- **minus** $\beta \cdot \text{KL}(\pi_\theta \,||\, \pi_{\text{ref}})$ — pulls back toward the SFT model's distribution

Assembling the full RLHF objective (continued)

- The advantage A itself is typically reward minus a learned **value function** $V(s)$, reducing variance (the same idea Lecture 10 covers in general RL)
- Three moving parts — policy, reward signal, KL anchor — all have to be balanced for training to stay stable

Reward hacking, in practice

- Reward models are imperfect proxies for human judgment
- A policy optimized hard enough can find outputs that score high on r_ϕ but are not actually better
- Symptoms: excessive length, sycophancy, repetitive hedging that raters happened to rate well
- The KL penalty, RM ensembling, and periodic re-collection of human data all mitigate this

Two concrete reward-hacking examples

- **Length hacking**: raters mildly preferred longer, more thorough-looking answers in the training data; the RM learns “longer = better” as a shortcut, and the RL policy drifts toward padding every answer
- **Confident-sounding hacking**: raters mildly preferred assertive phrasing over hedged phrasing; the RL policy learns to **sound** certain even on questions where it should express uncertainty
- Neither pattern was intended by the labelers — both emerged from a reward model generalizing a shallow correlation instead of the underlying quality judgment

Reward over-optimization: Goodhart's law in action

- "When a measure becomes a target, it ceases to be a good measure" — Goodhart's law applies directly to RLHF
- Early in RL fine-tuning, true quality (as judged by humans) and reward-model score rise together
- Push training far enough, and the two curves **diverge**: reward score keeps climbing while true human-judged quality plateaus or falls
- This is the empirical signature of reward hacking, and why practitioners track held-out human evaluation, not just RM score

Mitigating reward hacking

- **KL penalty**: keeps the policy from drifting too far from language the RM was actually trained on
- **Reward model ensembling**: average several independently trained RMs so idiosyncratic quirks of any one model matter less
- **Periodic re-collection**: gather fresh human preferences on the **current** policy's outputs and retrain the RM
- **Red-teaming**: deliberately search for prompts where the policy exploits the RM, and patch the training data

Multiple rounds of RLHF

- RLHF is rarely a single pass: as the policy improves, its outputs move into territory the reward model has never scored
- Practical pipelines iterate: deploy → collect fresh preferences on the newest policy's outputs → retrain the RM → run RL again
- Each round tightens the reward model's coverage of the policy's **actual** output distribution, reducing the reward-hacking gap
- This iterative loop is part of why frontier alignment pipelines are expensive to run repeatedly

RLHF: what it costs

- Three separately trained artifacts: SFT model, reward model, RL policy
- PPO requires a **value network** alongside the policy — another model to train and store
- RL training is notoriously sensitive to hyperparameters and can be unstable
- This complexity is exactly what the next method tries to remove

Check your understanding

- **What does the Bradley-Terry model convert a reward gap into?** — a probability of preference, via the sigmoid function
- **Why does PPO clip its objective?** — to prevent a single update from moving the policy too far based on one lucky (or unlucky) sample
- **What does the KL penalty term protect against?** — reward hacking, by anchoring the policy near the reference/SFT model
- Next: can we get RLHF-like behavior without training a separate reward model or running RL at all?



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Direct Preference Optimization (DPO)

Skipping the reward model

Motivation: do we need the RL loop at all?

- RLHF's complexity comes from training a separate reward model, then running online RL against it
- **DPO**'s idea: can we optimize the policy **directly** on preference data, with a simple supervised-style loss?
- No reward model to train, no PPO rollouts, no value network

Recap: what the RLHF objective actually optimizes

- RLHF's RL step finds the policy π_θ that maximizes expected reward $r_\phi(x, y)$, minus $\beta \cdot \mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$
- This has a well-defined mathematical optimum — for a **given** reward function r, there is a specific best policy
- DPO's insight starts here: if we know the **shape** of that optimal policy, we can work backward from it

Step 1: the optimal policy has a closed form

- For the RLHF objective above, the optimal policy $\pi^*(y \mid x)$ is proportional to $\pi_{\text{ref}}(y \mid x) \cdot \exp\left(r(x, y)/\beta\right)$
- In words: the best policy up-weights responses the reference model already assigns some probability to, boosted by how much reward they earn, and cooled by β
- This is a standard result from the theory of KL-regularized reward maximization — the course will not derive it from scratch, but will use its shape

Step 2: expressing reward via the policy

- Solve the Step-1 relationship for the reward instead of the policy:

$$r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \text{const}$$

- In words: the reward implied by a policy is proportional to how much more likely that policy makes a response, relative to the reference model
- This is the key move — reward is now written entirely in terms of a policy's log-probability ratio, no separate reward network required

Substituting into Bradley-Terry: the DPO loss

- Plug this reparameterized reward into the Bradley-Terry preference probability from the RLHF section
- The reward model terms cancel into log-probability-ratio terms for the policy being trained
- What remains is a loss purely over (prompt, preferred response, dispreferred response) triples — no reward model appears anywhere in the final expression

The DPO optimization objective

- Minimize over the policy π_θ, with data $\mathcal{D}$ of triples (x, y_w, y_l):

$$\mathcal{L}_{\text{DPO}}(\theta) = -\,\mathbb{E}_{(x,\,y_w,\,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

- y_w is the preferred response, y_l the dispreferred one, σ the sigmoid, and β the same KL-strength knob as in RLHF
- Each log-ratio is an implicit reward, so this is Bradley-Terry maximum likelihood with no reward model

The DPO loss, at a glance

- Directly increases $\log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x)$, each measured **relative to** π_{ref}
- Mathematically equivalent to fitting the Bradley-Terry reward implied by π_θ vs. π_{ref}
- Trained with ordinary supervised-style gradient descent — no sampling from the policy during training
- One stable training run replaces the RM-then-PPO pipeline

DPO trades RLHF's three-stage pipeline for a single fine-tuning objective on paired preference data.

What the DPO loss rewards and penalizes

Suppose for prompt x, response y_w is preferred over y_l.

- If π_θ already assigns y_w much higher relative probability than π_{ref} does, and y_l much lower, the loss is already small
- If π_θ assigns y_w and y_l similar probability (no preference expressed yet), the loss is large — strong gradient pushes y_w up and y_l down
- If π_θ has it backward (prefers y_l), the loss is largest — the strongest correction signal
- The loss's magnitude naturally scales with how wrong the current policy's implicit preference is

The DPO training loop, step by step

1. Sample a batch of (prompt, preferred response, dispreferred response) triples from the preference dataset
2. Compute log-probabilities of both responses under the **current policy** π_θ
3. Compute log-probabilities of both responses under the **frozen reference model** π_{ref}
4. Form the DPO loss from the two log-probability ratios, and backpropagate — no generation, no rollouts, no reward model forward pass

DPO vs. PPO: what each buys you

	DPO	PPO / RLHF
Reward model	not needed	required, trained separately
Value network	not needed	required
Training style	offline, supervised-style	online, RL rollouts
Stability	generally more stable	sensitive to hyperparameters
Non-preference signals	no — limited to given pairs	yes — any reward function

When would you still prefer PPO-style RL over DPO?

- DPO is **offline**: it only ever compares to the fixed preference pairs it was given, and cannot explore beyond that dataset during training
- PPO-style RL is **online**: the policy generates its own candidate responses during training and gets scored live, so it can discover behaviors the original preference data never covered
- This matters most when the reward signal is **verifiable** rather than just preferential — exactly the setting of the next section

Check your understanding

- **What does DPO remove from the RLHF pipeline?** — the separate reward model, the online rollouts, and the value network
- **Where does the "reward" go in the DPO loss, mathematically?** — reparameterized entirely in terms of the policy's log-probability ratio to the reference model
- **What is DPO's key limitation relative to PPO?** — it is offline, so it can't explore beyond the given preference pairs
- Next: what if the reward doesn't need a human or a learned model at all?



STEVENS
INSTITUTE OF TECHNOLOGY
1870

RL with Verifiable Rewards (RLVR)

Rewards a checker can verify

When the reward can be checked automatically

- RLHF's reward model is a **learned, noisy** proxy for human judgment
- Some tasks have a reward that can be **verified directly**, no human or learned model needed
  - Math: does the final answer match the ground truth?
  - Code: does the generated program pass the test suite?
- These **verifiable rewards** are cheap, exact, and don't drift or get hacked the way a learned RM can

Examples of verifiable rewards across domains

Domain	The check	Reward
Math	does the final answer match the known solution?	1 if correct, 0 otherwise
Code	does the program pass a held-out test suite?	fraction of tests passed
Structured extraction	does the output match the expected schema and fields?	exact-match or field-level score
Games / puzzles	do the moves satisfy the rules and reach a goal?	rule-based score

RL with Verifiable Rewards (RLVR)

- Same RL fine-tuning idea as RLHF, but $r(x, y)$ is a deterministic checker instead of a neural reward model
- No reward-model training stage at all
- Removes an entire source of reward hacking: there is nothing to hack, the checker is the ground truth
- Well suited to domains with an objective notion of “correct”

RLVR vs. RLHF: where does the reward come from?

	RLHF	RLVR
Reward source	learned reward model, fit to human preferences	deterministic checker (tests, answer matcher)
RM training needed	yes	no
Susceptible to RM hacking	yes	no (checker itself can still be gamed)
Best suited for	open-ended quality, tone, taste	tasks with objective correctness

Why this fits reasoning models

- Long chain-of-thought reasoning has a natural verifiable target: the final answer
- Sampling many candidate reasoning paths per problem and reinforcing the correct ones directly rewards **useful** reasoning, not just fluent-looking reasoning
- This RLVR recipe underlies recent reasoning-focused post-training (e.g. math and code specialists)
- Contrast with RLHF: no human rater is in this loop at all

Case examples: reasoning specialists

- Recent math-specialist and code-specialist models report large jumps in benchmark accuracy after an RLVR stage on top of SFT
- A common pattern: reasoning traces grow noticeably **longer** over training, as the policy discovers that more deliberate step-by-step reasoning raises its chance of a correct final answer
- Because the reward is purely outcome-based, the model is never directly told **which** reasoning steps were good — only that this path led to a right answer

Reward hacking is not just an RLHF problem

- A verifiable checker removes reward-**model** hacking, but the checker itself can still be gamed
- Example: a coding policy that special-cases the exact inputs used by a weak test suite, rather than solving the general problem
- Example: a math policy that pattern-matches to a memorized answer format rather than reasoning correctly, if training answers leak structure
- Verifiable rewards need to be as rigorous as the correctness notion they are meant to check

Check your understanding

- **What makes a reward "verifiable"?** — it can be computed by a deterministic checker, with no human or learned model in the loop
- **What does RLVR remove from the RLHF pipeline?** — the learned reward model, since a deterministic checker supplies the reward
- **Why can even a verifiable reward still be hacked?** — if the checker itself is incomplete or weak, a policy can exploit its blind spots
- Next: how do we evaluate a post-trained model?



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Evaluating post-trained models

Human eval · LLM-as-judge

Why evaluating “helpfulness” is hard

- Unlike a math answer, there is rarely one uniquely correct response to compare against
- Quality is multi-dimensional: correctness, tone, completeness, safety, formatting — a response can win on some and lose on others
- Different evaluators (and the same evaluator on different days) can disagree
- This is why post-training relies so heavily on **comparative** judgments rather than absolute scores

Human evaluation: pairwise comparison and ranking

- The same pairwise-comparison setup used to collect training data is also used to **evaluate** models: show raters two models' responses, ask which is better
- Aggregating many such comparisons across many models yields an **Elo-style ranking** — the same rating system used in chess, adapted to model comparisons
- Strength: directly measures what humans actually prefer. Weakness: slow and expensive to run at the scale needed to evaluate every training checkpoint

Automatic evaluation: LLM-as-judge

- Use a strong existing language model to **simulate** the human comparison step, then average or randomize to reduce noise
- Much faster and cheaper than human evaluation, so it can run continuously during development
- Known biases: judges can favor **longer** answers, favor whichever response appears first or second (position bias), or be fooled by confident phrasing
- Mitigations: randomize response order, average over multiple judge calls, periodically validate against human ratings

Benchmark suites used in practice

- Standardized sets of prompts, paired with either reference answers or a judging protocol, used to track alignment progress over time
- Cover a mix of instruction-following, reasoning, safety-refusal, and knowledge prompts
- Useful for tracking whether a change **helped or hurt** overall, but any single fixed benchmark can be gamed once its exact prompts are optimized against directly — the same Goodhart's-law risk seen earlier

Calibration and honesty

- A well-calibrated model's stated confidence should match its actual accuracy — if it says "I'm 90% sure," it should be right about 90% of the time
- Post-training can **hurt** calibration: RLHF in particular tends to make models sound more confident, since confident phrasing is often mildly preferred by raters
- Honesty evaluation asks a related but distinct question: does the model say "I don't know" when it doesn't, rather than fabricating a plausible-sounding but wrong answer?
- Both are active areas where post-training's incentives can pull against what we actually want

Check your understanding

- **Why is pairwise comparison used for evaluation, not just training?** — there's rarely one correct absolute answer, and comparison is more reliable than an absolute score
- **Name one known bias of LLM-as-judge evaluation.** — position bias, length bias, or being swayed by confident phrasing
- **What does "calibration" mean here, and how can post-training hurt it?** — confidence should match accuracy; preference-driven training can push toward sounding more confident than warranted



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Comparing the post-training landscape

Four methods, side by side

Putting it all together

Method	Data needed	Optimizes	Compute / models	Best suited for
SFT	ideal-response pairs	imitate demonstrations	cheapest — one model	bootstrapping instruction-following
RLHF (PPO)	pairwise preferences	RM score, online RL	most expensive — 4 models	open-ended quality, tone, safety
DPO	pairwise preferences	implicit reward via policy ratio	moderate — 2 models	RLHF-like gains, cheaper training
RLVR	a verifiable checker	exact task correctness	moderate — no reward model	math, code, checkable reasoning

A decision guide: which method, when?

- Start every pipeline with **SFT** — there is no substitute for first teaching the base model to follow instructions at all
- Target behavior is about open-ended quality, tone, or safety, and you have preference data → **RLHF or DPO**
- Need training as simple and stable as possible, willing to trade some online-exploration ability → prefer **DPO** over full RLHF/PPO
- Task has an automatic, objective correctness check → **RLVR**, no human labels needed

Modern assistants combine several stages

- In practice, a single shipped model is rarely the product of just one post-training method
- A typical recipe: SFT first, then RLHF or DPO for general quality/safety/tone, then RLVR on math- and code-heavy data to sharpen reasoning
- Some pipelines repeat stages iteratively, interleaving human-preference and verifiable-reward training

In practice, modern assistants combine several of these across post-training stages rather than picking just one.

Open problems in post-training

- **Scalable oversight**: as models tackle tasks humans can't easily verify (long proofs, large codebases), who — or what — judges the reward signal?
- **Reward hacking at scale**: both learned reward models and verifiable checkers can be gamed; more capable policies are better at finding the exploits
- **Calibration and honesty**: preference-driven training can push toward confident-sounding output over truthful uncertainty
- These are active research areas, not solved problems — a natural place to look for course project ideas

Recap and looking ahead

- Pretraining gives knowledge; post-training gives **behavior** — instruction-following, preference alignment, verified correctness
- The field has moved from RLHF's three-stage pipeline toward simpler (DPO) and more automatic (RLVR) alternatives
- Each method makes a different tradeoff between how much human signal it needs and how it optimizes that signal
- **Assignment 3** (search) is due today, October 15, at 3pm
- Next week: **LLM deployment** — parallelism, ZeRO, RAG, and agents, i.e. what happens after a model is post-trained

Post-training decides what the model does with what pretraining taught it.

Glossary: new terms from today

- **SFT**: supervised fine-tuning on (instruction, ideal response) pairs
- **Reward model (RM)**: a learned scalar function trained to predict human preference
- **Bradley-Terry model**: converts a reward gap into a preference probability via the sigmoid
- **PPO**: Proximal Policy Optimization; RL with a clipped objective for stable updates

Glossary: new terms from today (continued)

- **KL penalty**: keeps a fine-tuned policy close to its reference model
- **DPO**: Direct Preference Optimization; trains directly on preference pairs, no reward model
- **RLVR**: RL with Verifiable Rewards; the reward is a deterministic checker

Further reading pointers

- The original instruction-following/RLHF paper that established the SFT → reward model → PPO pipeline
- The DPO paper, for the full derivation this lecture only sketched
- Recent technical reports from reasoning-focused model releases that describe RLVR-style training in practice
- As with any paper for the course project: look for the sentence starting with “However” in the abstract — it usually names the gap the paper is trying to close



STEVENS
INSTITUTE OF TECHNOLOGY
1870

THANK **YOU**

Stevens Institute of Technology
1 Castle Point Terrace, Hoboken, NJ 07030